Automaton Auditor -- Interim Report

Date: February 2026
Repository: https://github.com/Heban-7/automaton-auditor

---

1. Executive Summary

The Automaton Auditor is a hierarchical multi-agent system built on LangGraph that performs autonomous code auditing using a "Digital Courtroom" metaphor. The system takes a GitHub repository URL and an optional PDF report as input, orchestrates parallel forensic investigations, runs dialectical judicial evaluation through three distinct judge personas, and synthesizes a structured audit report via deterministic conflict-resolution rules.

This interim submission covers the completed infrastructure, state definitions, forensic tool engineering, and the full detective layer with parallel fan-out/fan-in orchestration. The judicial layer (Prosecutor, Defense, Tech Lead) and the synthesis engine (ChiefJustice) are also implemented and wired into the graph, completing the end-to-end pipeline ahead of the final deadline.

Current Progress

Phase	Status
Infrastructure (pyproject.toml, .env, config)	Complete
State Definitions (Pydantic + TypedDict)	Complete
Forensic Tools (repo_tools, doc_tools)	Complete
Detective Layer (RepoInvestigator, DocAnalyst, VisionInspector)	Complete
Judicial Layer (Prosecutor, Defense, TechLead)	Complete
Synthesis Engine (ChiefJustice)	Complete
Graph Orchestration (dual fan-out/fan-in)	Complete
Audit Report Renderer	Complete

2. Architecture Decisions

2.1 Why Pydantic Over Plain Dicts

The project documentation mandates strict typing. We chose Pydantic `BaseModel` for all inter-agent data contracts and `TypedDict` for the graph state for the following reasons:

Validation at the boundary. Every `Evidence` object returned by a detective and every `JudicialOpinion` returned by a judge is a Pydantic model with constrained fields (e.g., `score: int = Field(ge=1, le=5)`, `confidence: float = Field(ge=0.0, le=1.0)`). This means malformed data is caught immediately at construction time rather than silently corrupting downstream nodes.

Structured output enforcement. LangChain's `.with_structured_output(JudicialOpinion)` requires a Pydantic schema. Without it, judges would return freeform text that would require fragile parsing. With the schema, the LLM is forced to produce valid JSON that is automatically deserialized into a typed object.

State reducers for parallelism. The `AgentState` uses `TypedDict` with `Annotated` reducers:

```
class AgentState(TypedDict):
    repo_url: str
    pdf_path: str
    rubric_dimensions: List[Dict]
    repo_path: str
    evidences: Annotated[Dict[str, List[Evidence]], operator.ior]
    opinions: Annotated[List[JudicialOpinion], operator.add]
    final_report: Optional[AuditReport]
```

- `operator.ior` on `evidences` merges dictionaries from parallel detectives (`{"repo": [...]}` + `{"doc": [...]}` + `{"vision": [...]}`) without any key overwriting the other.
- `operator.add` on `opinions` concatenates lists from parallel judges so all 30 opinions (3 judges x 10 dimensions) accumulate correctly.

Without reducers, the last-finishing parallel branch would silently overwrite the outputs of the earlier branches -- a critical data-loss bug that is invisible until you notice missing evidence in the final report.

Comparison with plain dicts:

Aspect	Pydantic/TypedDict	Plain dict
Field validation	Automatic at construction	Manual checks everywhere
IDE autocompletion	Full type hints	None
Structured LLM output	Direct `.with_structured_output()`	Requires manual JSON parsing
Parallel safety	Reducers prevent overwrites	Last-write-wins data loss
Serialization	`.model_dump()` / `.model_dump_json()`	Manual

2.2 AST Parsing Strategy

The RepoInvestigator must verify structural properties of Python code (e.g., "Does `src/graph.py` instantiate a `StateGraph`?", "Are there `add_edge` calls creating fan-out?"). We use Python's built-in `ast` module rather than regex for the following reasons:

Structural correctness. Regex cannot distinguish between `StateGraph` appearing in a comment, a string literal, and an actual class instantiation. AST parsing operates on the parsed syntax tree, so we only match genuine code constructs.

Implementation. We built a custom `_StructureVisitor(ast.NodeVisitor)` in `src/tools/repo_tools.py` that walks the full AST of every Python file in the cloned repository and collects four signal types:

1. **Classes** -- name, base classes, line number. This detects `BaseModel` and `TypedDict` inheritance.
2. **Imports** -- fully qualified import paths. This detects `operator`, `langgraph`, `pydantic` usage.
3. **Function calls** -- `ast.unparse(node.func)` captures the full call expression. This detects `StateGraph(...)`, `builder.add_edge(...)`, `.with_structured_output(...)`, `subprocess.run(...)`, `os.system(...)`.
4. **Attribute access** -- detects method chains and attribute patterns.

The `analyze_graph_structure()` function then aggregates these signals across all `.py` files into a structured findings dict:

```
findings = {
    "state_graph_found": True/False,
```

```
    "add_edge_calls": [...],
    "add_conditional_edges_calls": [...],
    "pydantic_models": [...],
    "typeddict_classes": [...],
    "operator_reducers": [...],
    "structured_output_calls": [...],
    "os_system_calls": [...],
    "tempfile_usage": True/False,
    "subprocess_usage": True/False,
}
```

Each detective evidence-collection function queries this findings dict to answer its specific forensic question without re-parsing.

2.3 Sandboxing Strategy

The auditor clones arbitrary GitHub repositories -- potentially containing malicious code. Our sandboxing strategy has three layers:

Layer 1: Temporary directory isolation. Every `clone_repo()` call creates a `tempfile.TemporaryDirectory(prefix="auditor_")`. The cloned code never touches the auditor's own working directory. The temporary directory is cleaned up when the handle goes out of scope or `.cleanup()` is called.

Layer 2: subprocess.run() with controls. We use `subprocess.run()` exclusively -- never `os.system()`. Every call includes:

- `capture_output=True` -- stdout/stderr are captured, not printed to the auditor's console
- `text=True` -- output is decoded as UTF-8
- `timeout=120` -- prevents indefinite hangs on large repositories
- Return code checking -- non-zero exits raise `RuntimeError` with the stderr message

Layer 3: No code execution. The AST parser uses `ast.parse()` which builds a syntax tree without executing any of the target repository's code. We never `import` or `exec` anything from the cloned repo.

```python
def clone_repo(url: str) -> tuple[str, tempfile.TemporaryDirectory]:
    tmp = tempfile.TemporaryDirectory(prefix="auditor_")
    repo_dir = os.path.join(tmp.name, "repo")
    result = subprocess.run(
        ["git", "clone", "--depth=50", url, repo_dir],
        capture_output=True, text=True, timeout=120,
    )
    if result.returncode != 0:
        tmp.cleanup()
        raise RuntimeError(f"git clone failed: {result.stderr.strip()}")
```

```
    return repo_dir, tmp
```

2.4 PDF Parsing with Fallback Chain

The DocAnalyst needs to parse PDF reports which may vary widely in format. We implemented a fallback chain:

1. **Primary: docling** -- The project documentation recommends docling. It converts the PDF to Markdown and we split into paragraph-level chunks for RAG-lite querying.
2. **Fallback: PyMuPDF (fitz)** -- If docling fails (dependency issues, unsupported PDF format), we fall back to PyMuPDF which extracts page-level text.
3. **Error reporting** -- If both fail, a clear `RuntimeError` is raised and the DocAnalyst records an `Evidence(found=False)` object explaining the failure.

Image extraction for the VisionInspector uses PyMuPDF's `page.get_images()` API to extract embedded images, save them to a temporary directory, and pass paths to the multimodal LLM.

---

3. StateGraph Architecture

3.1 High-Level Flow

The complete StateGraph implements a dual fan-out/fan-in architecture with conditional error handling:

```
START
  |
  v
ContextBuilder  (loads rubric.json into state)
  |
  +---> RepoInvestigator  (clones repo, runs 7 forensic protocols)
  |
  +---> DocAnalyst         (parses PDF, checks theoretical depth +
accuracy)
  |
  +---> VisionInspector    (extracts images, analyses diagrams via vision
LLM)
  |
  v
EvidenceAggregator  (fan-in: validates completeness, fills gaps)
  |
  |  [conditional edge: if zero evidence -> skip to ReportRenderer]
```

```
  |
  +---> Prosecutor   ("Trust No One" -- adversarial lens)
  |
  +---> Defense      ("Reward Effort" -- optimistic lens)
  |
  +---> TechLead     ("Does it work?" -- pragmatic lens)
  |
  v
ChiefJustice  (deterministic conflict resolution)
  |
  v
ReportRenderer  (serializes AuditReport to Markdown)
  |
  v
END
```

3.2 Detailed StateGraph Diagram

```
                        +------------------+
                        |      START       |
                        +--------+---------+
                                 |
                                 v
                        +------------------+
                        | ContextBuilder   |
                        | (load rubric)    |
                        +--------+---------+
                                 |
                  +----------------+----------------+
                  |                |                 |
                  v                v                 v
        +-----------------+ +-----------+ +------------------+
        |RepoInvestigator | |DocAnalyst | |VisionInspector   |
        |                 | |           | |(implementation   |
        |7 forensic       | |2 forensic | | required,        |
        |protocols via    | |protocols  | | execution         |
        |AST + git log    | |via PDF    | | optional)         |
        +---------+-------+ +-----+-----+ +--------+---------+
                  |               |                 |
                  +---------------+-----------------+
                                  |
                                  v
                        +-------------------+
                        |EvidenceAggregator |
                        |(validate + fill   |
```

```
                | missing dims)     |
                +---------+---------+
                          |
          [conditional: has_evidence?]
                          |
        +---------------+----------------+
        |               |                |
        v               v                v
   +------------+ +------------+ +------------+
   | Prosecutor | |  Defense   | |  TechLead  |
   | Score: 1-5 | | Score: 1-5 | | Score: 1-5 |
   | (harsh)    | | (generous) | | (pragmatic)|
   +------+-----+ +------+-----+ +------+-----+
        |               |               |
        +---------------+---------------+
                        |
                        v
                +-------------------+
                |   ChiefJustice    |
                | (deterministic    |
                |  Python rules)    |
                |                   |
                | Rules applied:    |
                | 1.Security Override|
                | 2.Fact Supremacy  |
                | 3.Func. Weight    |
                | 4.Variance Reeval |
                +---------+---------+
                          |
                          v
                +-------------------+
                |  ReportRenderer   |
                | (Markdown file)   |
                +---------+---------+
                          |
                          v
                  +-------+--------+
                  |      END       |
                  +----------------+
```

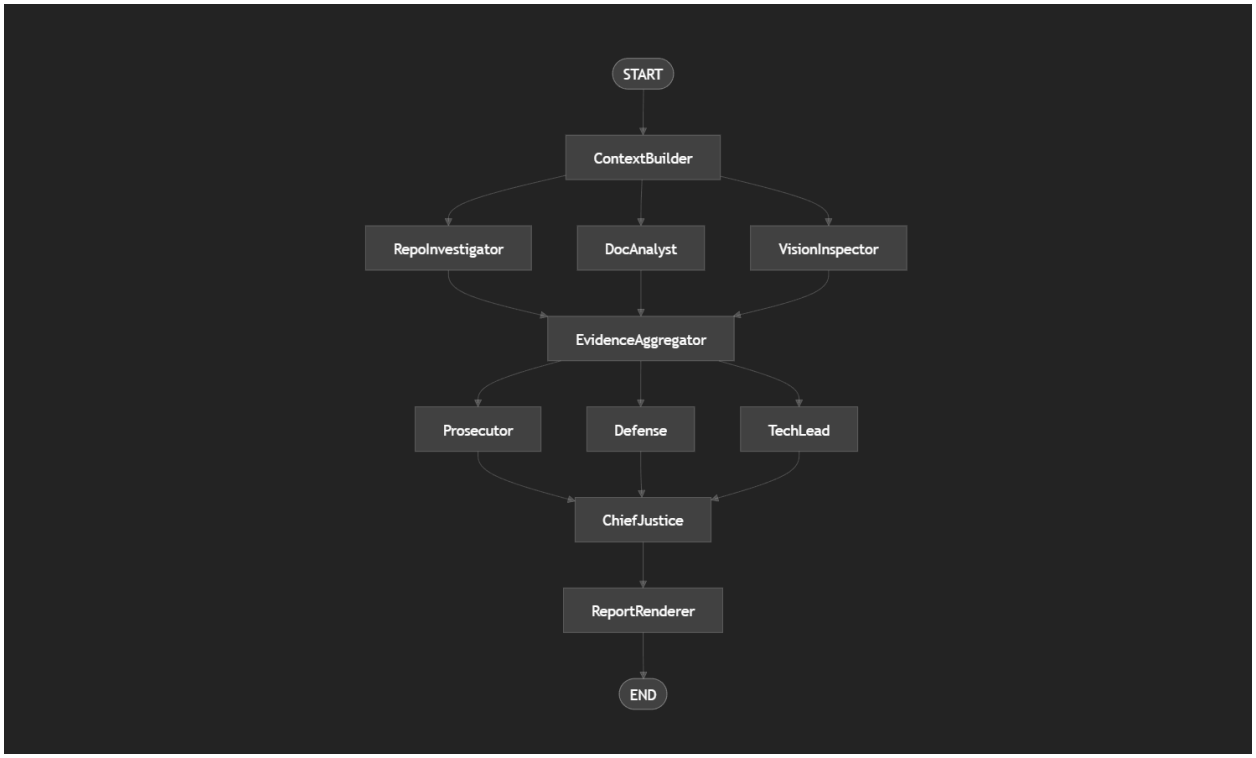

Fig 1: StateGraph Diagram

3.3 Fan-Out / Fan-In Mechanics

Fan-Out is implemented by adding multiple edges from a single source node. LangGraph executes all target nodes concurrently:

```
# Detective fan-out: three detectives run in parallel
builder.add_edge("context_builder", "repo_investigator")
builder.add_edge("context_builder", "doc_analyst")
builder.add_edge("context_builder", "vision_inspector")
```

Fan-In is implemented by adding edges from all parallel branches to a single synchronization node. LangGraph waits for all branches to complete before executing the sync node:

```
# Detective fan-in: aggregator waits for all three
builder.add_edge("repo_investigator", "evidence_aggregator")
builder.add_edge("doc_analyst", "evidence_aggregator")
builder.add_edge("vision_inspector", "evidence_aggregator")
```

Conditional routing uses `add_conditional_edges` with a function that returns a list of target nodes:

```
builder.add_conditional_edges(
    "evidence_aggregator",
    _check_evidence,  # returns ["prosecutor","defense","tech_lead"] or
["report_renderer"]
    ["prosecutor", "defense", "tech_lead", "report_renderer"],
)
```

3.4 State Synchronization

The key challenge in parallel execution is preventing data loss. Our `AgentState` uses two reducers:

- **`evidences: Annotated[Dict[str, List[Evidence]], operator.ior]`** `-- Each detective writes to a different key ("repo", "doc", "vision"). The operator.ior reducer merges these dicts via |= so all evidence is preserved.`
- **`opinions: Annotated[List[JudicialOpinion], operator.add]`** `-- Each judge returns a list of 10 opinions. The operator.add reducer concatenates all lists so the ChiefJustice receives all 30 opinions (3 judges x 10 dimensions).`

`Without these reducers, only the last-finishing parallel branch's output would survive in state.`

---

4. Detective Layer -- Forensic Protocols

4.1 RepoInvestigator

The RepoInvestigator handles 7 rubric dimensions targeting `github_repo`:

Dimension	Forensic Method
Git Forensic Analysis	`git log --oneline --reverse` parsed for commit count, timestamps, progression keywords
State Management Rigor	`AST scan for BaseModel/TypedDict classes, operator.add/operator.ior reducers`
Graph Orchestration	`AST scan for StateGraph, add_edge,`

	`add_conditional_edges with parallelism heuristics`
Safe Tool Engineering	`AST scan for tempfile.TemporaryDirectory, subprocess.run, absence of os.system`
Structured Output	`AST scan for .with_structured_output() and .bind_tools() calls`
Judicial Nuance	`Prompt extraction and comparison for distinct personas`
Chief Justice Synthesis	`Source scan for deterministic rule keywords (security_override, fact_supremacy, etc.)`

Each protocol outputs a typed `Evidence` object with `goal`, `found`, `content`, `location`, `rationale`, and `confidence` fields.

4.2 DocAnalyst

The DocAnalyst handles 2 rubric dimensions targeting `pdf_report`:

Dimension	Forensic Method
Theoretical Depth	Keyword search for "Dialectical Synthesis", "Fan-In/Fan-Out", "Metacognition", "State Synchronization" with context-length heuristic to distinguish substantive explanations from buzzword dropping
Report Accuracy	Regex extraction of file paths from PDF text, cross-referenced against the cloned repo's actual file listing. Produces `verified` vs `hallucinated` path lists

4.3 VisionInspector

The VisionInspector handles 1 rubric dimension targeting `pdf_images`:

Dimension	Forensic Method
Architectural Diagram Analysis	PyMuPDF image extraction, base64 encoding, multimodal LLM analysis (GPT-4o) with structured JSON response for diagram

	classification and parallel-flow detection

Per the project documentation: "You may implement this feature, but running it to get results is optional." The implementation is complete; execution depends on image availability in the PDF.

---

5. Judicial Layer and Synthesis Engine

5.1 Judge Implementation

All three judges share a common execution pattern but diverge fundamentally in their system prompts:

Common pattern:

```
llm = ChatOpenAI(model="gpt-4o",
temperature=0.2).with_structured_output(JudicialOpinion)

for dimension in rubric["dimensions"]:
    evidence = flatten_and_filter(state["evidences"], dimension["id"])
    opinion = llm.invoke([SystemMessage(system_prompt),
HumanMessage(evidence + criteria)])
    opinions.append(opinion)
```

Persona differentiation:

Judge	Philosophy	Legal Statute	Scoring Tendency
Prosecutor	"Trust No One. Assume Vibe Coding."	Statute of Orchestration: linear graphs = "Orchestration Fraud" (score 1); freeform judge output = "Hallucination Liability" (cap at 2)	Harshest defensible score
Defense	"Reward Effort and Intent."	Statute of Effort: broken graph + good AST logic = boost to 3; LLM synthesis + distinct personas = partial credit 3-4	Most generous defensible score
Tech Lead	"Does it actually	Statute of	Realistic 1/3/5;

	work?"	Engineering: dict state = "Technical Debt" (score 3); unsandboxed cloning = "Security Negligence" (overrides effort)	tie-breaker

Structured output enforcement: Every LLM call uses `.with_structured_output(JudicialOpinion)` which forces the model to return valid JSON matching the Pydantic schema. A retry loop (up to 2 retries) handles transient parsing failures.

5.2 ChiefJustice Synthesis Engine

The ChiefJustice uses **deterministic Python logic** -- not an LLM prompt. The four named rules are applied in priority order:

1. **Security Override** -- If the Prosecutor's argument contains security-related keywords (`"os.system"`, `"shell injection"`, `"unsanitized"`), the final score is capped at 3 regardless of other judges' scores.
2. **Fact Supremacy** -- If the Defense claims high merit (score > 2) but all detective evidence for that criterion shows `found=False`, the Defense is overruled and the score is capped at 3.
3. **Functionality Weight** -- The Tech Lead's score carries the highest weight in the weighted average:
    a. Architecture criteria: TechLead 0.55, Prosecutor 0.25, Defense 0.20
    b. Other criteria: TechLead 0.45, Prosecutor 0.30, Defense 0.25
4. **Variance Re-evaluation** -- When score variance exceeds 2 (e.g., Prosecutor=1, Defense=5), a dissent summary is generated documenting both arguments, and the weighted score is computed with the above weights rather than a simple average.

5.3 Report Renderer

The final `AuditReport` is serialized to a structured Markdown file containing:

- **Executive Summary** -- overall score with per-criterion PASS/FAIL tags
- **Criterion Breakdown** -- 10 sections, each with the final score, all three judge opinions with cited evidence, dissent summary (where applicable), and remediation instructions
- **Remediation Plan** -- sorted by score (lowest first), with file-level instructions for each failing criterion

---

6. Known Gaps and Remaining Work

6.1 Addressed Since Interim Plan

The following items that were originally planned for the final submission have already been implemented:

- Judicial layer with three distinct judge personas (Prosecutor, Defense, TechLead)
- ChiefJustice with hardcoded deterministic conflict resolution rules
- Report renderer producing structured Markdown
- Full end-to-end graph with dual fan-out/fan-in
- Conditional edges for error handling (skip judges when no evidence)

6.2 Remaining for Final Submission

Gap	Priority	Plan
End-to-end runtime testing	High	Run the swarm against the auditor's own repository and a peer's repository to produce `audit/report_onself_generated/` and `audit/report_onpeer_generated/`
LangSmith trace verification	High	Verify that `LANGCHAIN_TRACING_V2=true` captures the full detective -> judge -> synthesis chain in the LangSmith dashboard
MinMax feedback loop	Medium	After receiving peer audit results, update the agent to catch issues the peer's agent identified; document changes in the final report reflection section
Dockerfile	Low	Create a containerized runtime for reproducibility
Video demonstration	Medium	Record screen capture showing the full audit workflow end-to-end
VisionInspector live test	Low	Test diagram analysis against a real PDF with embedded architecture diagrams

6.3 Error Handling Improvements

- The DocAnalyst's cross-referencing currently depends on `repo_path` being in state. If the RepoInvestigator fails to clone, the DocAnalyst cannot cross-reference. A future improvement would add a conditional edge that retries the clone or skips cross-referencing gracefully.
- The VisionInspector's multimodal LLM call lacks structured output enforcement (it parses raw JSON from the model response). Adding `.with_structured_output()` with a Pydantic schema would improve reliability.

---

7. File Manifest

File	Purpose
`pyproject.toml`	Dependencies managed via uv
`.env.example`	Required API keys template
`rubric.json`	Machine-readable 10-dimension rubric with forensic instructions, success/failure patterns, and synthesis rules
`src/config.py`	Loads .env, rubric.json; provides dimension filtering by target_artifact
`src/state.py`	Pydantic models (Evidence, JudicialOpinion, CriterionResult, AuditReport) and AgentState TypedDict with operator.ior / operator.add reducers
`src/tools/repo_tools.py`	Sandboxed git clone, git history extraction, AST-based structural analysis
`src/tools/doc_tools.py`	PDF ingestion (docling + PyMuPDF fallback), keyword search, file-path extraction, image extraction
`src/nodes/detectives.py`	RepoInvestigator, DocAnalyst, VisionInspector nodes + EvidenceAggregator
`src/nodes/judges.py`	Prosecutor, Defense, TechLead with distinct system prompts and .with_structured_output(JudicialOpinion)

`src/nodes/justice.py`	ChiefJustice with deterministic rules + Markdown report renderer
`src/graph.py`	Full StateGraph with dual fan-out/fan-in, conditional edges, CLI entry point

---

8. Conclusion

The Automaton Auditor's core architecture is complete. The system implements the full Digital Courtroom pipeline: forensic evidence collection through specialized detectives, dialectical evaluation through three judge personas with fundamentally different philosophies, and deterministic synthesis through a ChiefJustice that applies named rules (Security Override, Fact Supremacy, Functionality Weight, Variance Re-evaluation) rather than delegating to an LLM. The remaining work focuses on runtime validation, peer audit execution, and documentation refinement for the final submission.